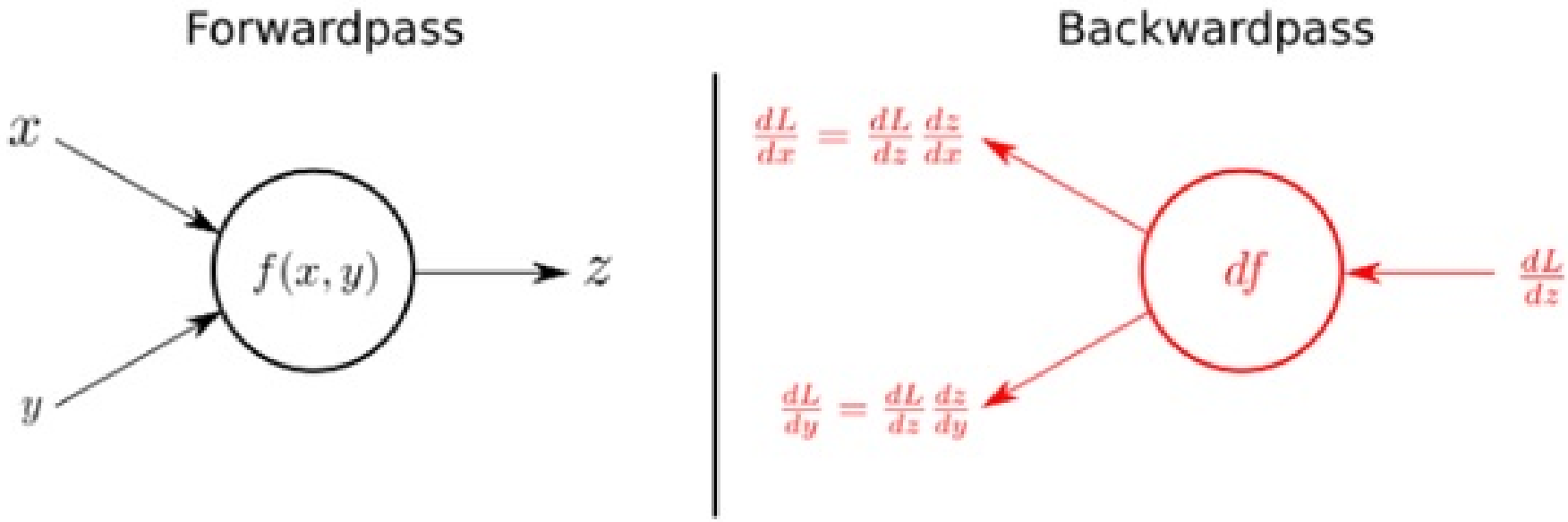
Forward Pass vs. Backward Pass



Forward Pass

- The processing from input layer to hidden layer(s) and then to the output layer is called forward propagation.
- The sum(input*weights)+bias is applied at each layer and then the activation function value is propagated to the next layer.
- The next layer can be another hidden layer or the output layer.

Loss Function

- Simplest loss function

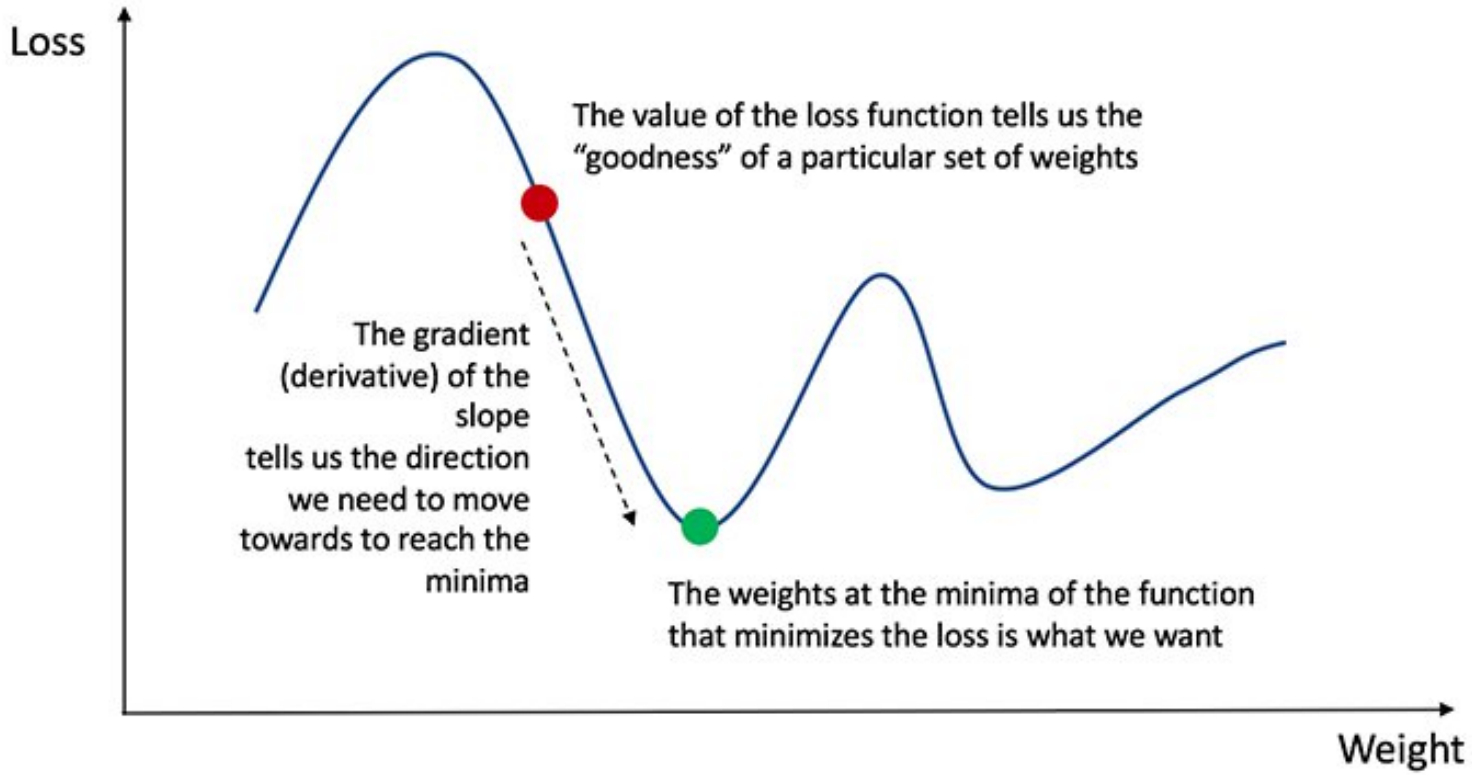
$$Sum - of - Squares\ Error = \sum_{i=1}^{n} (y - \hat{y})^2$$

- Goal: to find the best set of weights and biases that minimizes the loss function.

Backward Pass

- Once the output is arrived at, at the last layer (the output layer), the error is computed (the predicted output minus the original output).
- This error is required to correct the weights and biases used in forward propagation with the use of derivative functions.
- The gradient suggests how steeply the error will be reduced or increased for a change in the weight.

Gradient Descent

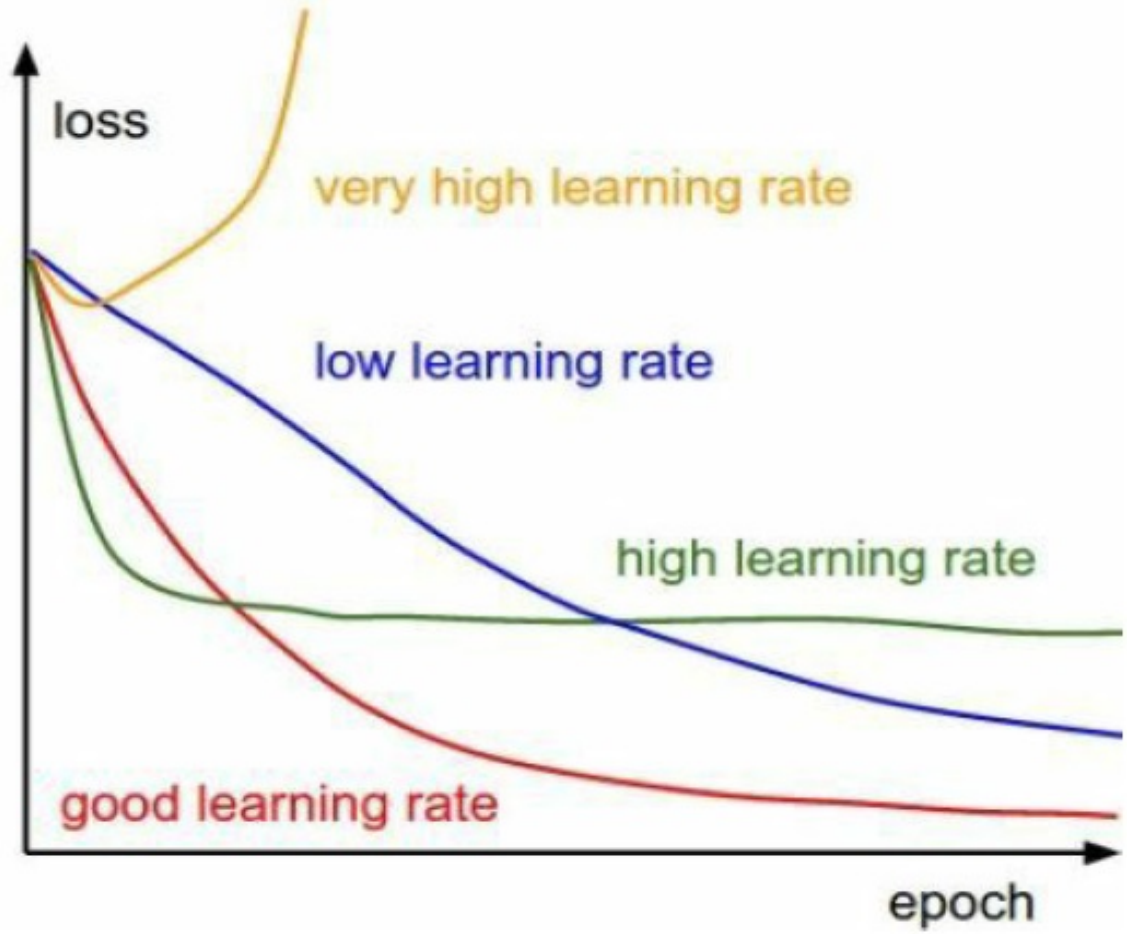


Learning Rate

- A scalar parameter, analogous to step size in numerical integration.
- Used to set the rate of adjustments to reduce the errors faster.
- Used in backpropagation during adjustment of weights and bias.

More the learning rate, the faster the algorithm will reduce the errors and **faster will be the training process.**

Learning Rate

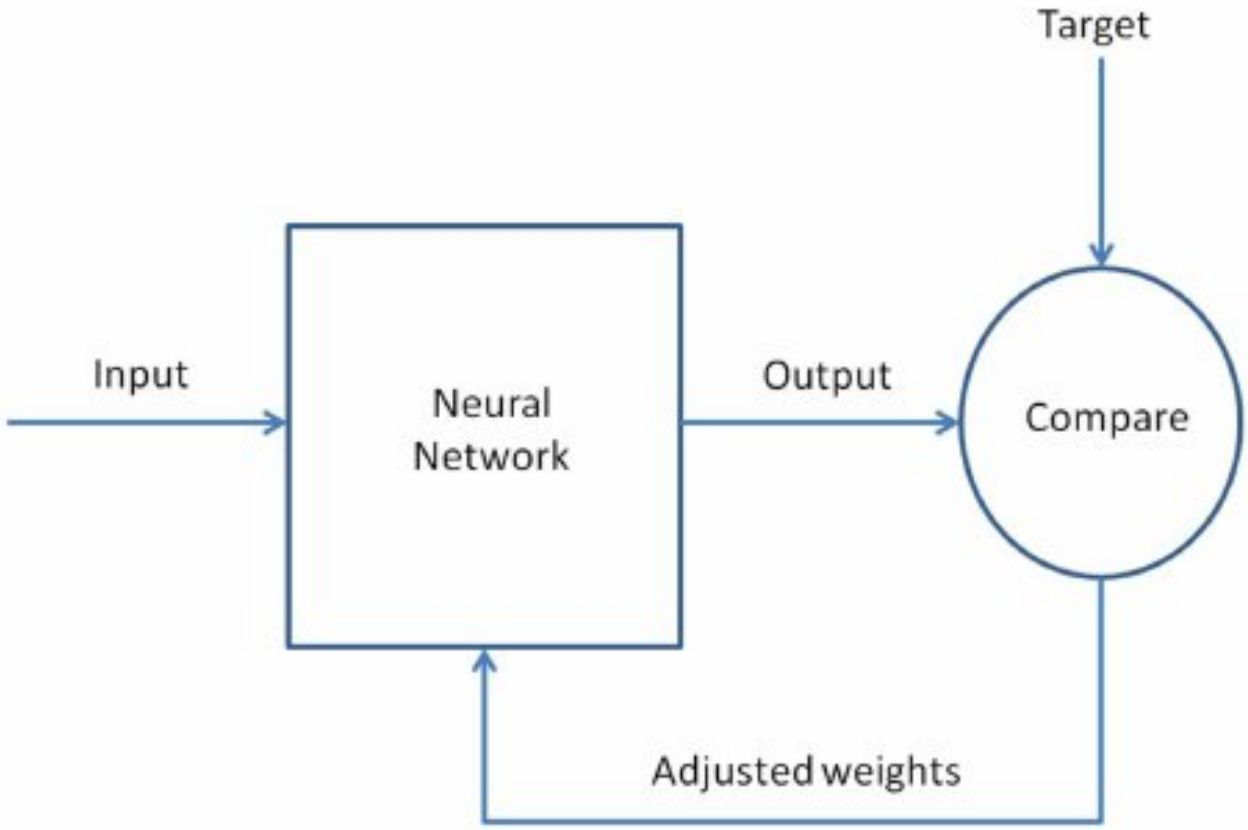


Epoch

- One iteration or pass through the process of providing the network with an input and updating the network's weights once.
- It is a full run of feed-forward followed by backpropagation to update of weights.
- It is also one full read through of the entire dataset.

Typically, **many epochs**, in the order of tens of thousands at times, are required to **train the neural network efficiently.**

ANN Terminology: Illustration



Steps: Forward Propagation

1. Take the input as a matrix.
2. Initialize the weights and biases with random values. This is one time and these will be updated with the error thorough out the propagation process.
3. Repeat the steps 4 to 9 for each training pattern (presented in random order), until the error is minimized.
4. Apply the inputs to the network.
5. Calculate the output for every neuron from the input layer, through the hidden layer(s), to the output layer.
6. Calculate the error at the outputs: actual minus predicted.

Steps: Backpropagation

7. Use the output error to compute error signals for previous layers. The partial derivative of the activation function is used to compute the error signals.

8. Use the error signals to compute weight adjustments.

9. Apply the weight adjustments.

https://www.youtube.com/watch?v=GvQwE2OhL8I

MCQ

Back propagation is a learning technique that adjusts weights in the neural network by propagating weight changes.

(i) Forward from inputs to outputs.

(ii) Backward from outputs to inputs.

(iii) Forward from inputs to hidden nodes.

(iv) Backward from outputs to hidden nodes.

(v) None of the above.

MCQ

What's the main point of difference between human & machine intelligence?

(i) Human have emotions and machines do not.

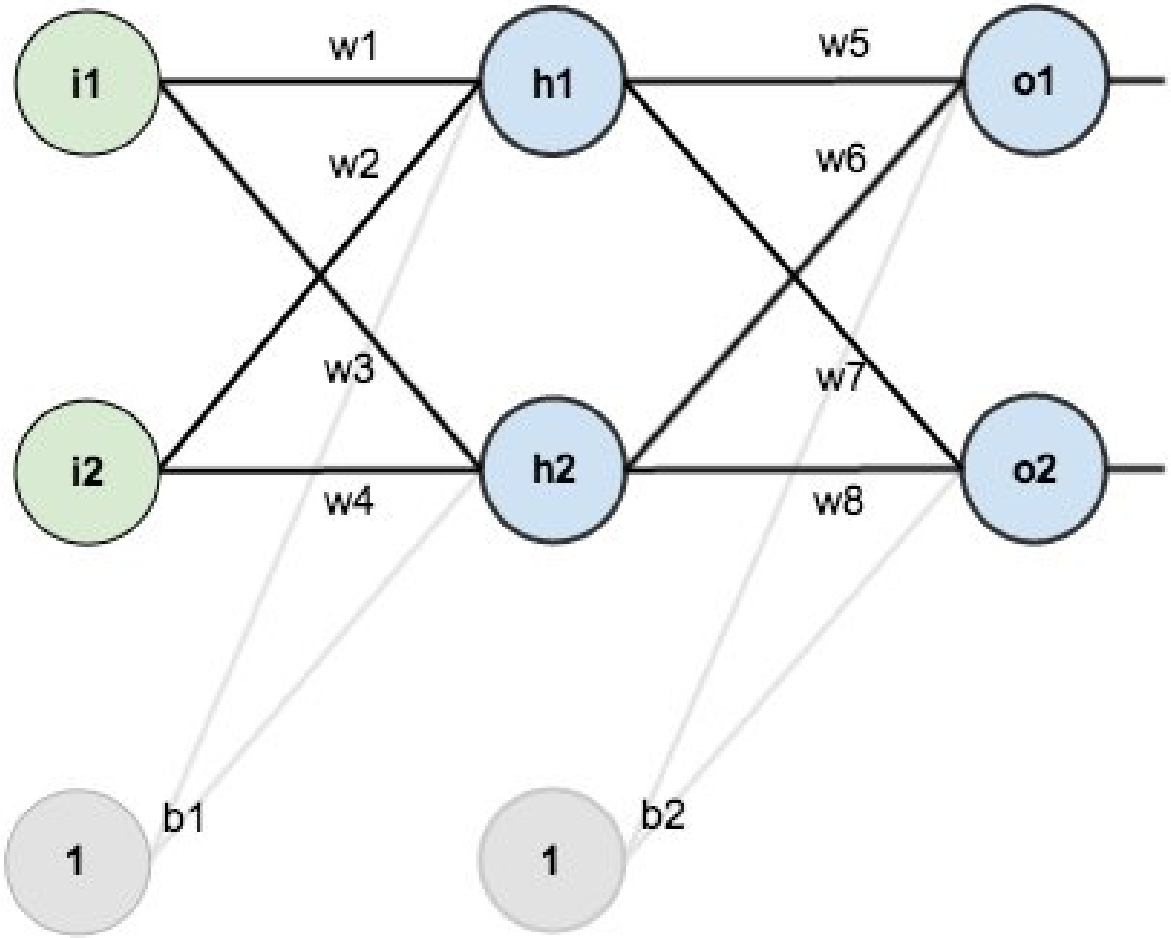
(ii) Human have more IQ & intellect that the machines.

(iii) Human perceive everything as a pattern while machine perceive it merely as data.

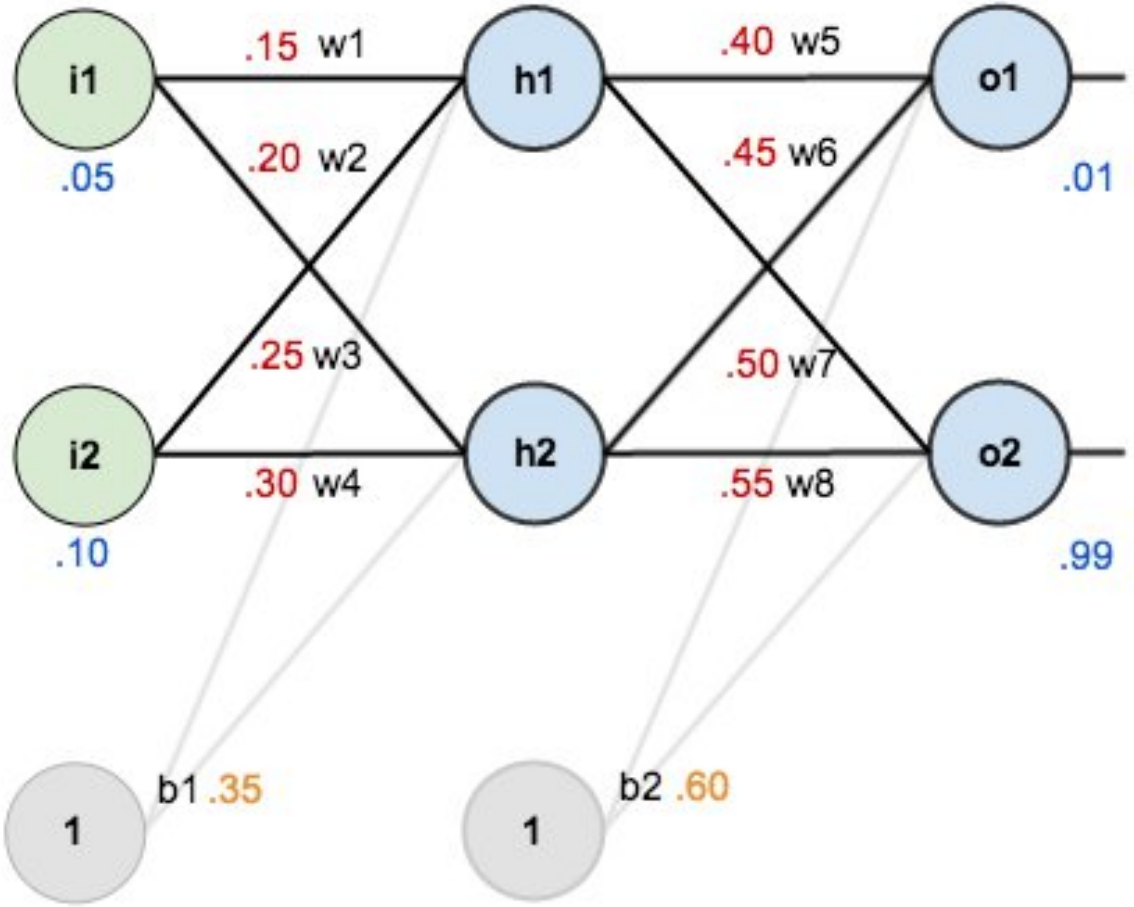
(iv) Human have sense organs unlike machines.

(v) None of the above

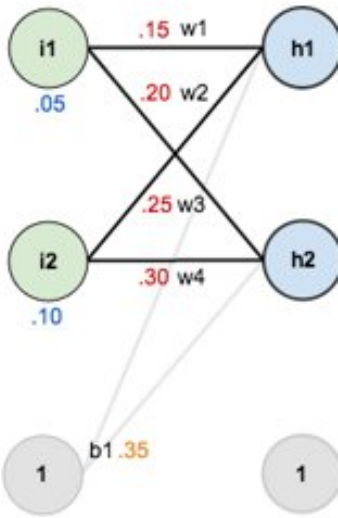
Step by step ANN training example



Step by step ANN training example



Step by step ANN training example



Forward Pass:

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

We then squash it using the logistic function to get the output of h_1:

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

Carrying out the same process for h_2 we get:

$$out_{h2} = 0.596884378$$

Step by step ANN training example

Output Layer:

Here's the output for o_1:

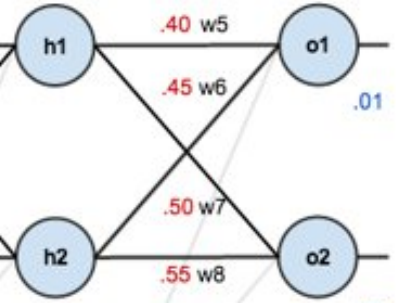
$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

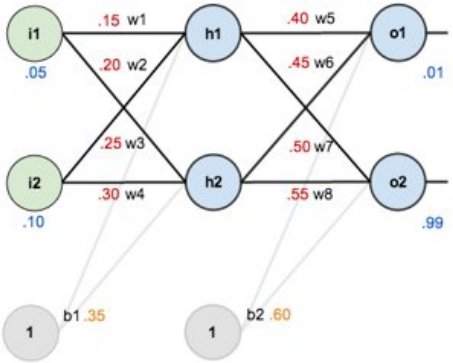
$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

And carrying out the same process for $o_2$we get:

$$out_{o2} = 0.772928465$$



Step by step ANN training example



Calculating the Total Error

$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

For example, the target output for o_1 is 0.01 but the neural network output 0.75136507, therefore its error is:

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

Repeating this process for o_2 (remembering that the target is 0.99) we get:

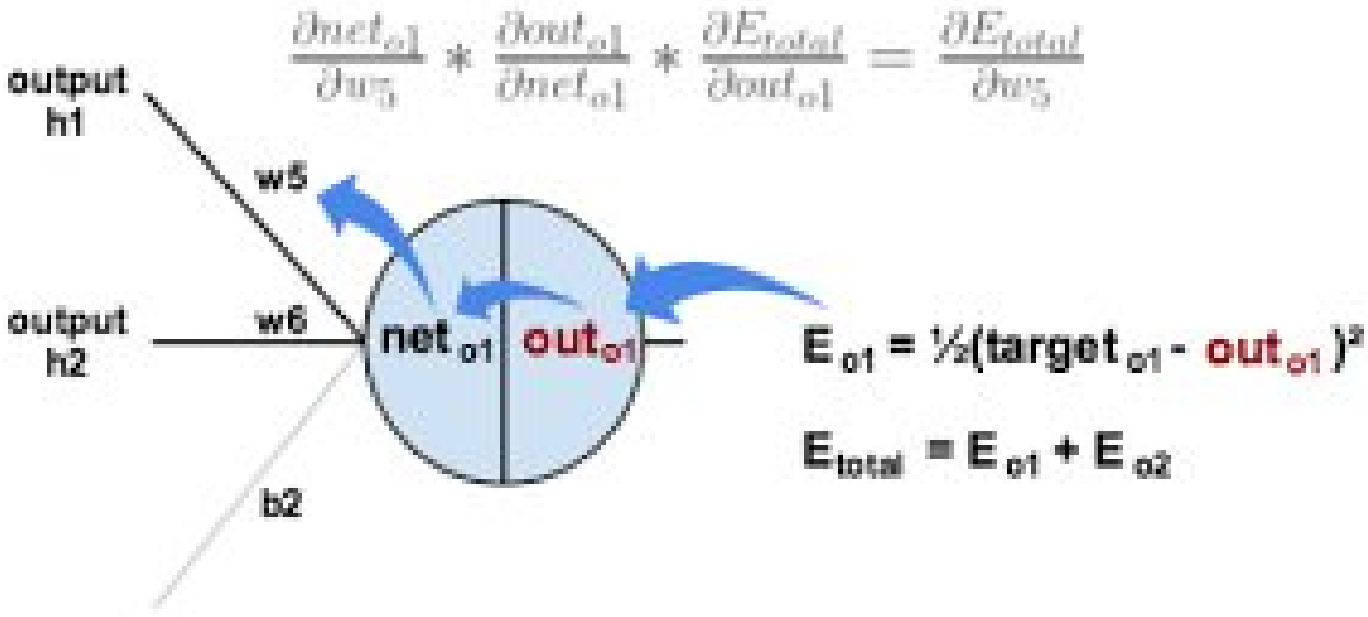
$$E_{o2} = 0.023560026$$

The total error for the neural network is the sum of these errors:

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

Step by step ANN training example

The Backward Pass: How much change in W5 effects the total error.



Chain Rule

The Backward Pass: How the chain rule works

Partial derivative of the Error

$$E_{total} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

Partial derivative of the logistic function

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

The Backward Pass: How the chain rule works

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

Putting it altogether

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

To decrease the error, we then subtract the above calculated value from the current weight subjected to the learning rate.

$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

Repeat the same process for each node in the output layer.

$$w_6^+ = 0.408666186$$

$$w_7^+ = 0.511301270$$

$$w_8^+ = 0.561370121$$

UPDATED WEIGHTS

The Backward Pass: For Hidden Layer

$$w_2^+ = 0.19956143$$

$$w_3^+ = 0.24975114$$

$$w_4^+ = 0.29950229$$

UPDATED WEIGHTS

We have updated all the weights!!!

Learning Process

Initial error on the network = 0.298371109

After the first round of backpropagation,
The error = 0.291027924

After repeating this process about 10,000 times,
The error = 0.0000351085

At this point (after 10,000 iterations), when we feed forward 0.05 and 0.1, the two outputs neurons generate 0.015912196 (vs 0.01 target) and 0.984065734 (vs 0.99 target)

MCQ

Which of the statements given above is true?

Statement 1: It is possible to train a neural network well by initializing all the weights as 0

Statement 2: It is possible to train a neural network well by initializing biases as 0

Statement 3: It is not possible to train a neural network by randomly initializing all the weights.

(i) Statement 1 is true while Statement 2 is false.

(ii) Statement 2 is true while statement 1 is false.

(iii) Statement 1 and 2 are false while statement 3 is true.

(iv) All statements are true.

(v) All statements are false.

ACTIVATION FUNCTION

- Decide whether a neuron should be activated or not
- Non linear transformation performed over the input signal.
- The transformed output is then send to the next layer of neurons as input.

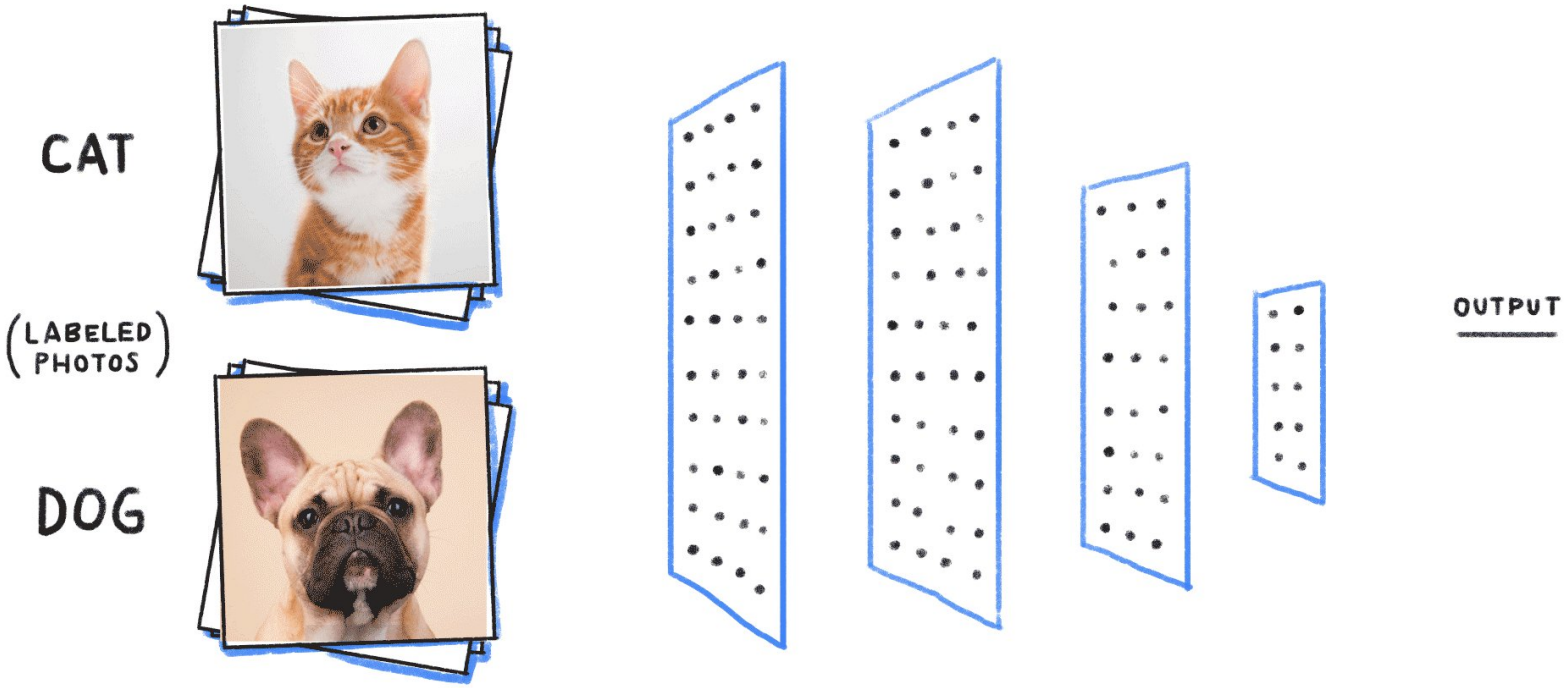
LOSS FUNCTION

- The group of functions that are minimized in the ANN model fitting procedure

- Measure of how good a prediction model does in terms of being able to predict the expected outcome

- Need to optimize the network – through a loss function

LOSS FUNCTION – Intuition



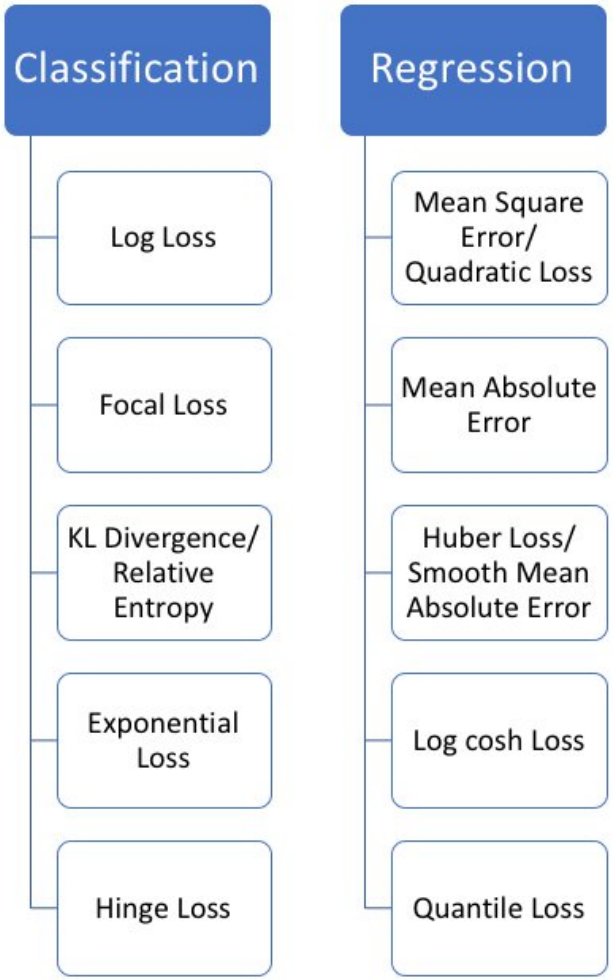
LOSS FUNCTION

There is not a single Loss Function that works for all kind of data.

Depends on,

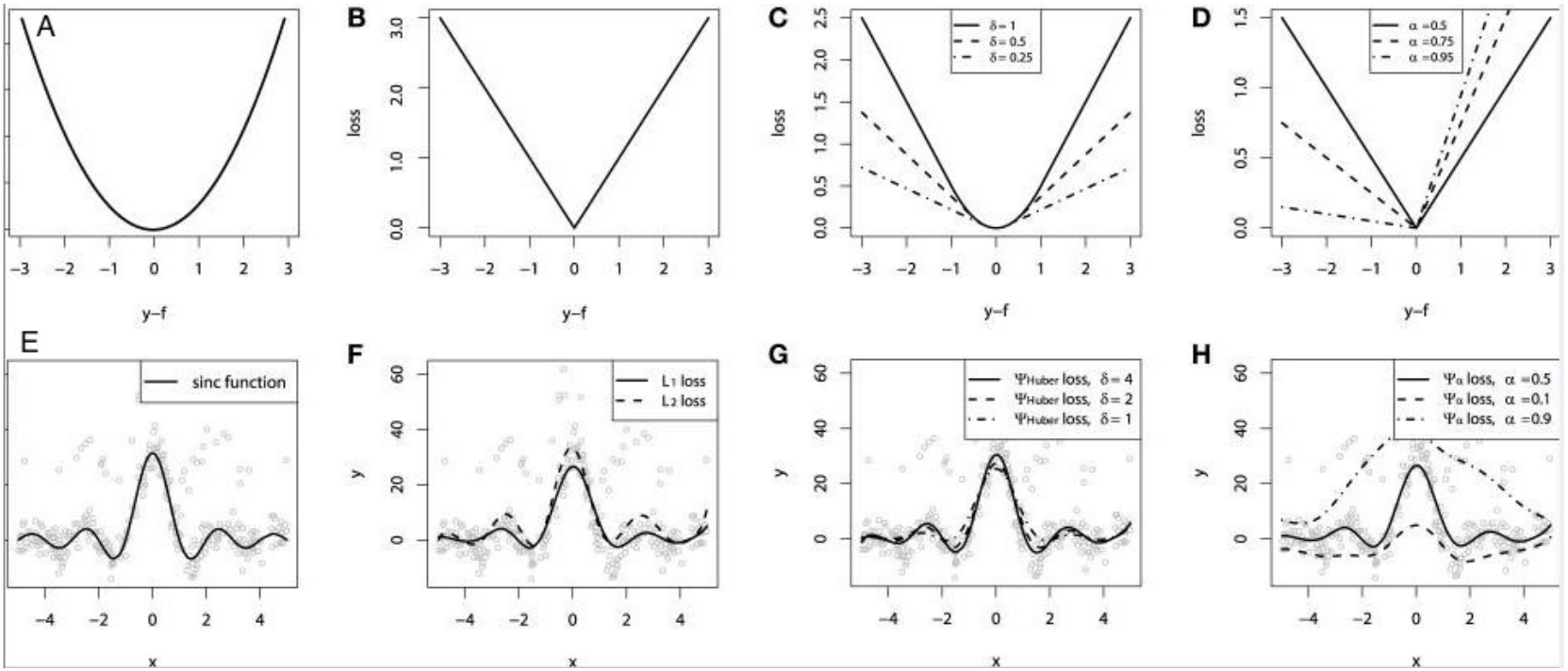
- presence of outliers
- algorithm
- time efficiency of gradient descent
- ease of finding the derivatives
- ease of finding confidence of predictions

LOSS FUNCTIONS - Categorization



https://medium.com/@phuctrt/loss-functions-why-what-where-or-when-189815343d3f

LOSS FUNCTIONS



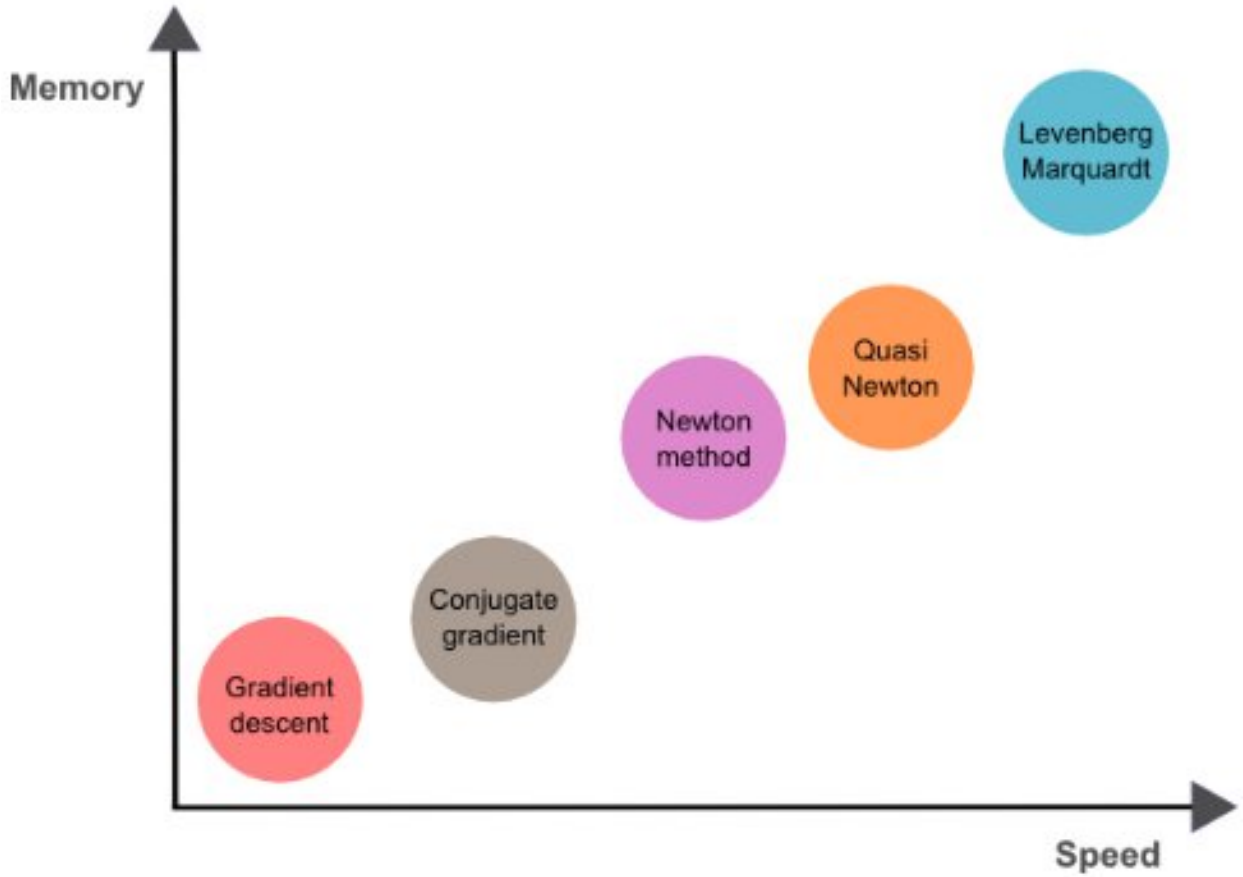
Continuous loss functions: (A) MSE loss function; (B) MAE loss function; © Huber loss function; (D) Quantile loss function. Demonstration of fitting a smooth GBM to a noisy sinc(x) data: **(E)** original sinc(x) function; **(F)** smooth GBM fitted with MSE and MAE loss; **(G)** smooth GBM fitted with Huber loss with δ = {4, 2, 1}; **(H)** smooth GBM fitted with Quantile loss with α = {0.5, 0.1, 0.9}.

https://medium.com/@phuctrt/loss-functions-why-what-where-or-when-189815343d3f

ANN TRAINING ALGORITHMS

- The procedure used to carry out the learning process in a neural network.
- The learning problem is formulated in terms of the minimization of a loss function, f.
- Loss function – Composed of an error and a regularization terms.

ANN TRAINING ALGORITHMS



https://www.neuraldesigner.com/blog/5_algorithms_to_train_a_neural_network